How do you solve a problem with no known answer?

Models have read everything we've written. The problems left **can't be looked up**. You have to discover the answer.

scientific discovery | **open models** | **~$500** / problem

OUR ANSWER keep training the model **at test time**, on the **one** problem in front of it.

Yuksekgonul · Koceja · Li · Bianchi · McCaleb · Wang · Kautz · Choi · Zou · Guestrin · Sun

Federico Bianchi

Staff Scientist at Together AI, previously OpenEvidence and Stanford. I work on **self-improving agents** and **scientific discovery**.

TextGrad → **self-improving AI systems** → Nature 2025

Agents4Science → **first AI-authored conference** → Nature Biotechnology

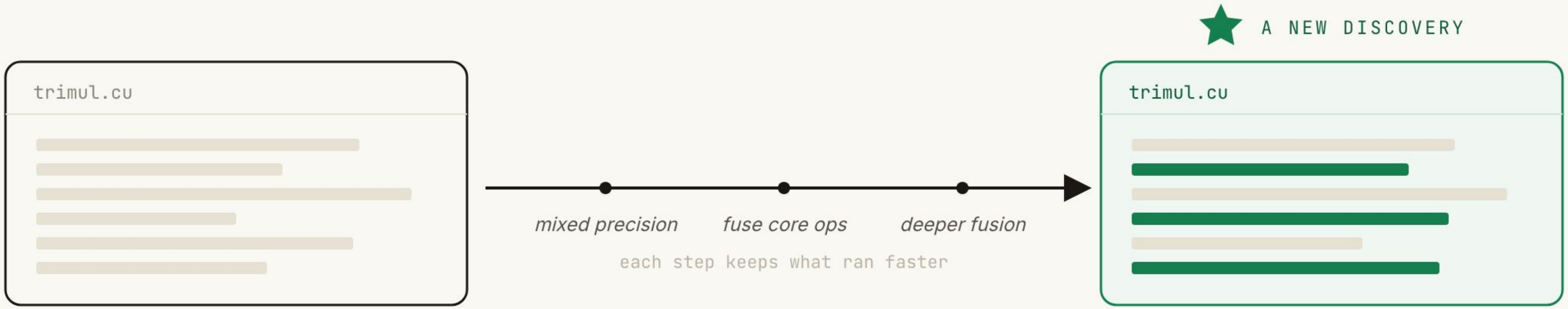
EinsteinArena → **platform for open math discovery** → 11 new SOTA

The next stage of AI.

From **recalling** what we know to **discovering** what we don't.

What a discovery actually looks like.

Start from a slow kernel. End faster than any human ever wrote.



unoptimized baseline

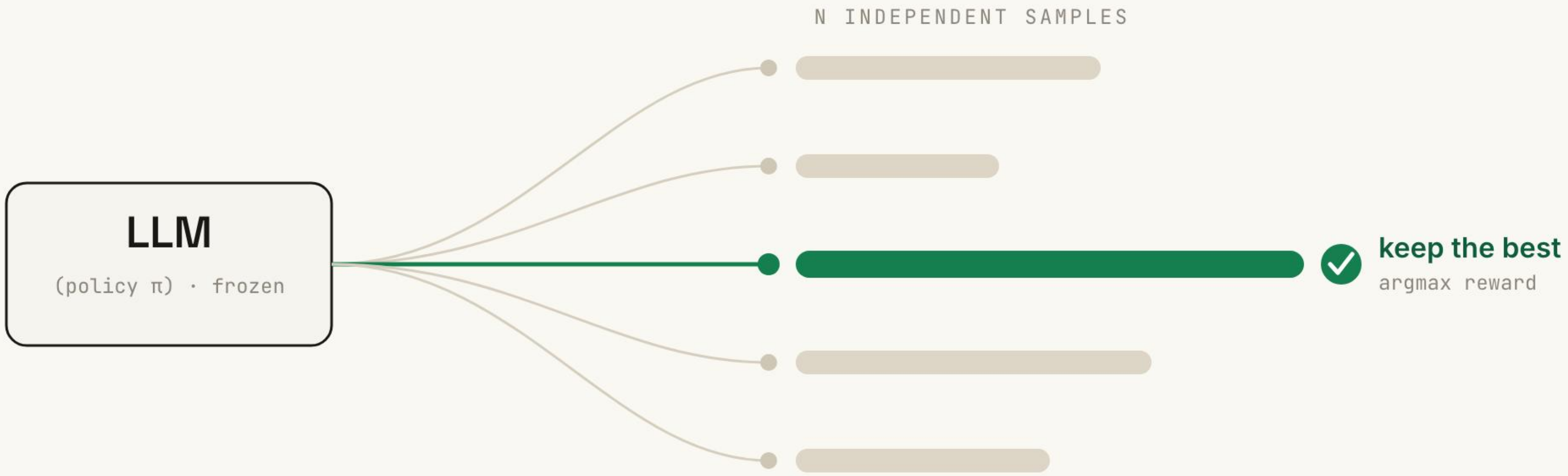
5352 µs

faster than any human wrote

1161 µs · 4.6× faster

The simplest way to search.

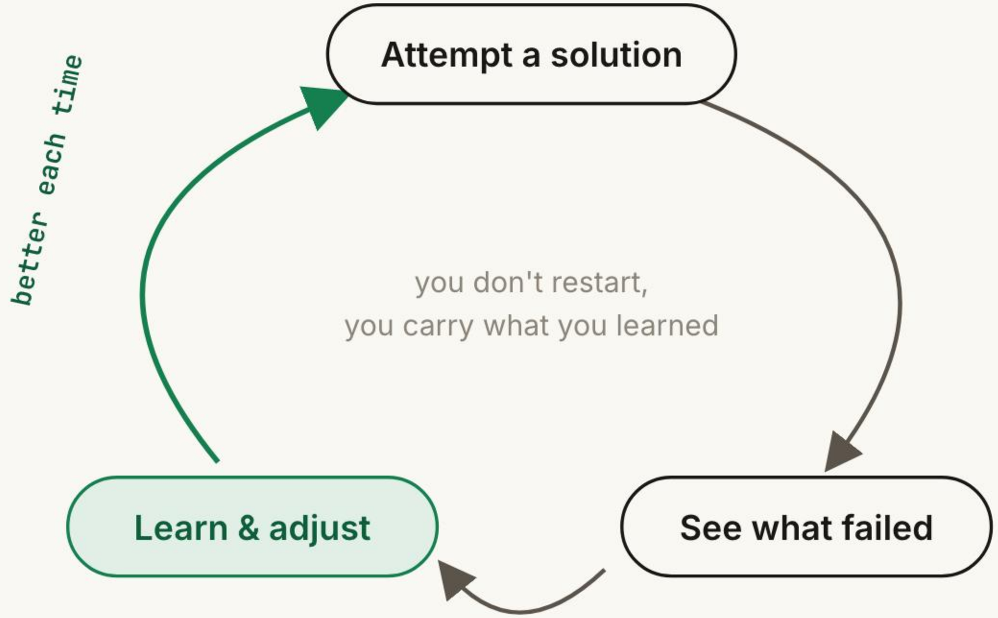
Sample many solutions from a frozen model, then keep the best one. That's **Best-of-N**.



THE CATCH every sample comes from the **same frozen model**. It **never learns** between attempts.

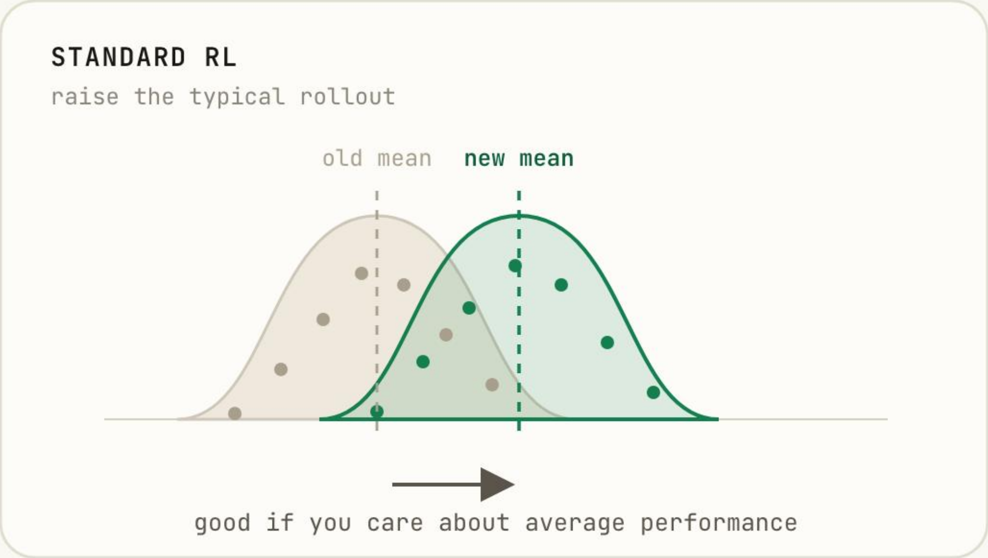
No one solves a hard problem on the first try.

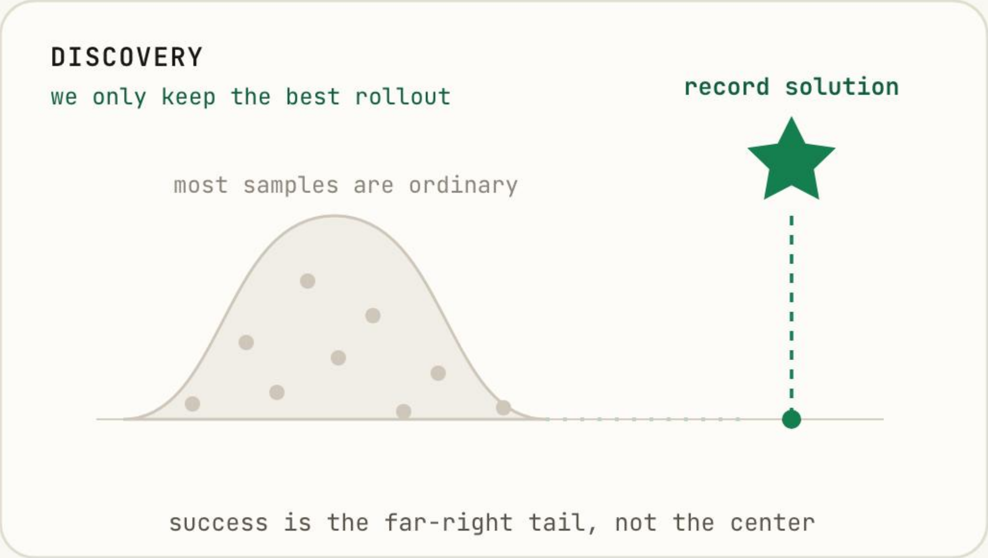
We try, get stuck, see what failed, and come back sharper.



The catch: RL hill-climbs toward the average, not the best.

a policy samples many possible solutions
each dot is one rollout · x-position is reward





reward of a sampled solution → better

RL maximizes the average reward. Discovery keeps only the best sample, so the max is what counts.

Reinforcement learning at test time, on a single problem. Optimize for the best result, not the average.

The policy is disposable. We don't need a model that generalizes, just **one artifact** that beats the state of the art.

The change: from average reward to best-sample reward.

STANDARD RL

maximize

$\mathbb{E}_{y\sim\pi\theta}[R(y)]$

pushes up the mean rollout

TTT-DISCOVER

maximize

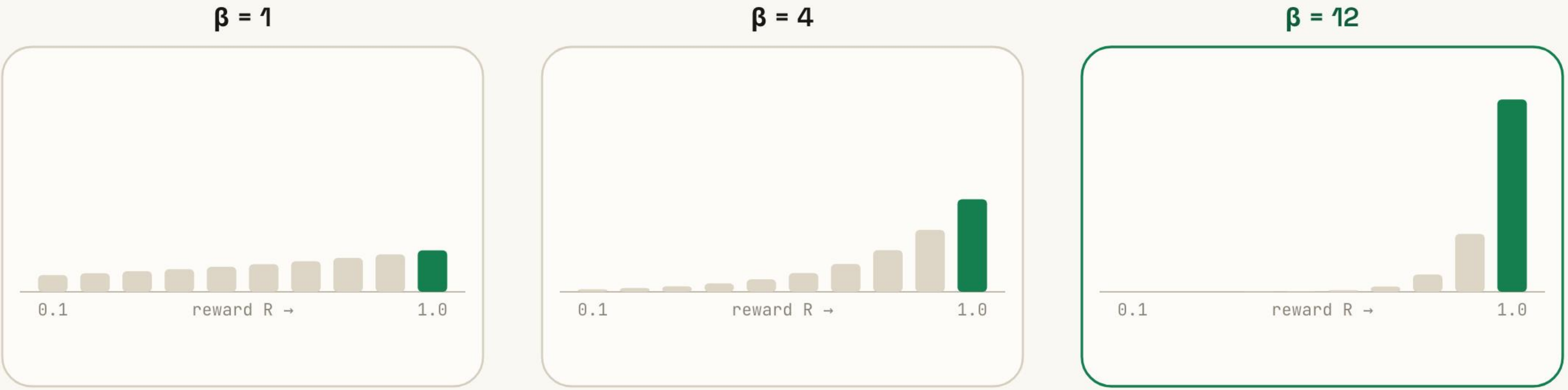
$\log\ \mathbb{E}_{y\sim\pi\theta}[e^{\beta R(y)}]$

tracks the max as β grows

x = one problem · R(y) = verifier score

open model · ~$500 / problem

Turn up β, and the weight piles onto the best rollout.



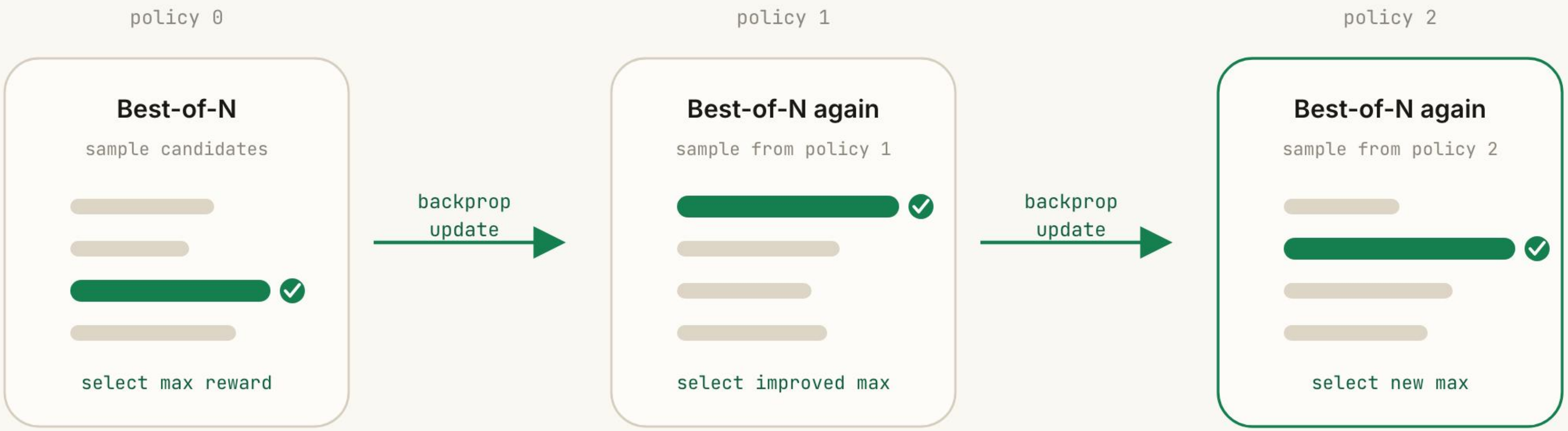
close to uniform

focusing on top rewards

almost only the best

$w(y) = e^{\beta R(y)} / \Sigma\, e^{\beta R}$, a soft-max over rewards. As $\beta \to \infty$ it tracks the max, not the mean.

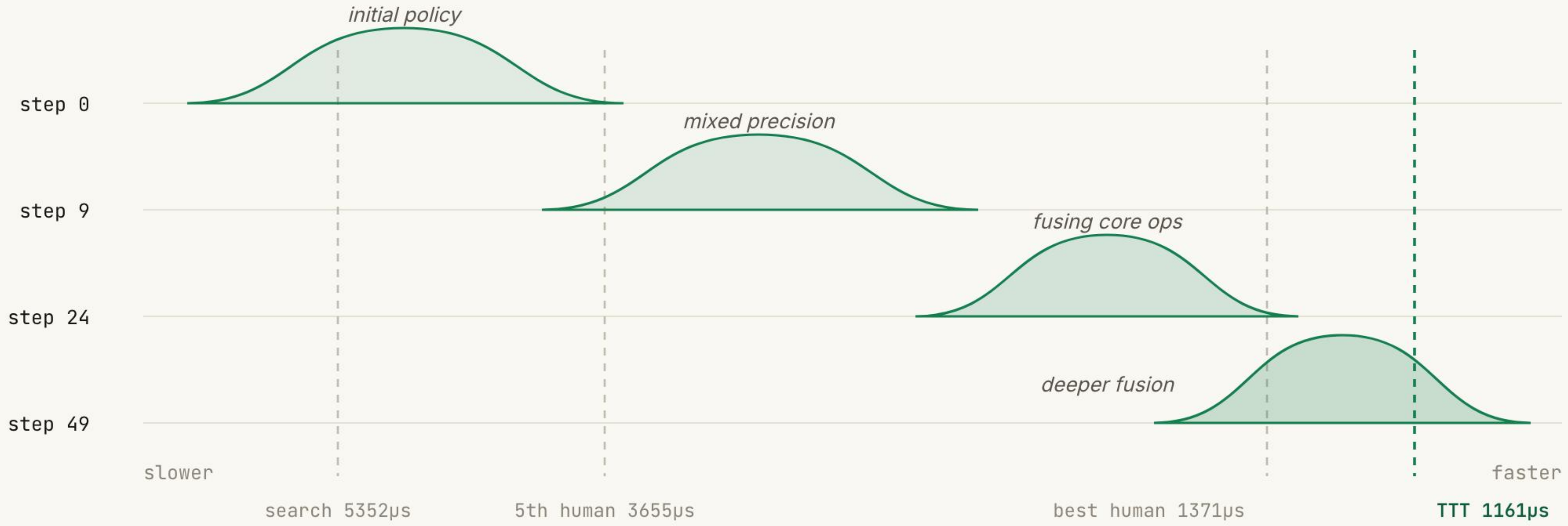
Updates shift the sampling distribution.



Not more guesses from one frozen model.
Each update shifts the distribution of sampled solutions.

Each update shifts samples toward faster kernels.

TriMul kernel-runtime samples
same H100 task · lower μs is better



Past the best known solution, in every field.

GPU KERNELS · TRIMUL

runtime μs · lower is better

	1st human	TTT-Discover	reduction
A100	4531	2198	−51%
H100	1371	1161	−15%
B200	1039	914	−12%
MI300X	2516	1556	−38%

Trained on H100 · tested on A100, H100, B200, MI300X

MATH · ALGORITHMS · BIOLOGY

mathematics · lower ↓
Erdős overlap
0.380927
0.380876

mathematics · lower ↓
Autocorr. AC1
1.50314
1.50287

algorithms · higher ↑
AtCoder AHC039
566,997
567,062

biology · higher ↑
Single-cell denoising
0.64
0.71

Top number is best known; green number is TTT-Discover.

Checked by people who aren’t us.

> *"This is similar to the current best human solutions, but executed better. Most human solutions lag behind in fusing some of the more complex operators together, resulting in this solution outperforming them by a large margin."*
>
> GPUMode TriMul competition review

> *"It is straightforward to verify that the provided functions give bounds that improve upon the state of the art: one evaluates the quantity of interest over the discrete set of points and checks the norm constraints are satisfied."*
>
> Prof. Davide Torlo · Università di Roma La Sapienza

Open models can discover if they keep learning at test time.

Give the model one hard problem, a verifier, and a max-seeking objective.
The output is a **new artifact**: a kernel, a bound, an algorithm, or a denoiser.

`gpt-oss-120b · open` `~$500 / problem` `4 fields, 4 firsts` `trained on H100 · wins on every GPU`

test-time-training.github.io/discover